

BIAI PROJECT



Oskar Chłopaś
Piotr Nowak

Project goal

The goal of the project was to create a deep learning model capable of recognizing F1 race cars in images. The project code allows training such a model, saving it, evaluating its performance, and using it to classify new images. Transfer learning was utilized with several neural networks, and a dataset containing images of various Formula 1 teams was used.



Data

As input data, we used a database of nearly 4,000 images of F1 cars.



- AlphaTauri F1 car
- Ferrari F1 car
- McLaren F1 car
- Mercedes F1 car
- Racing Point F1 car
- Red Bull Racing F1 car
- Renault F1 car
- Williams F1 car

Libraries

TensorFlow – An open-source machine learning library created by Google. It is responsible for, among other things:

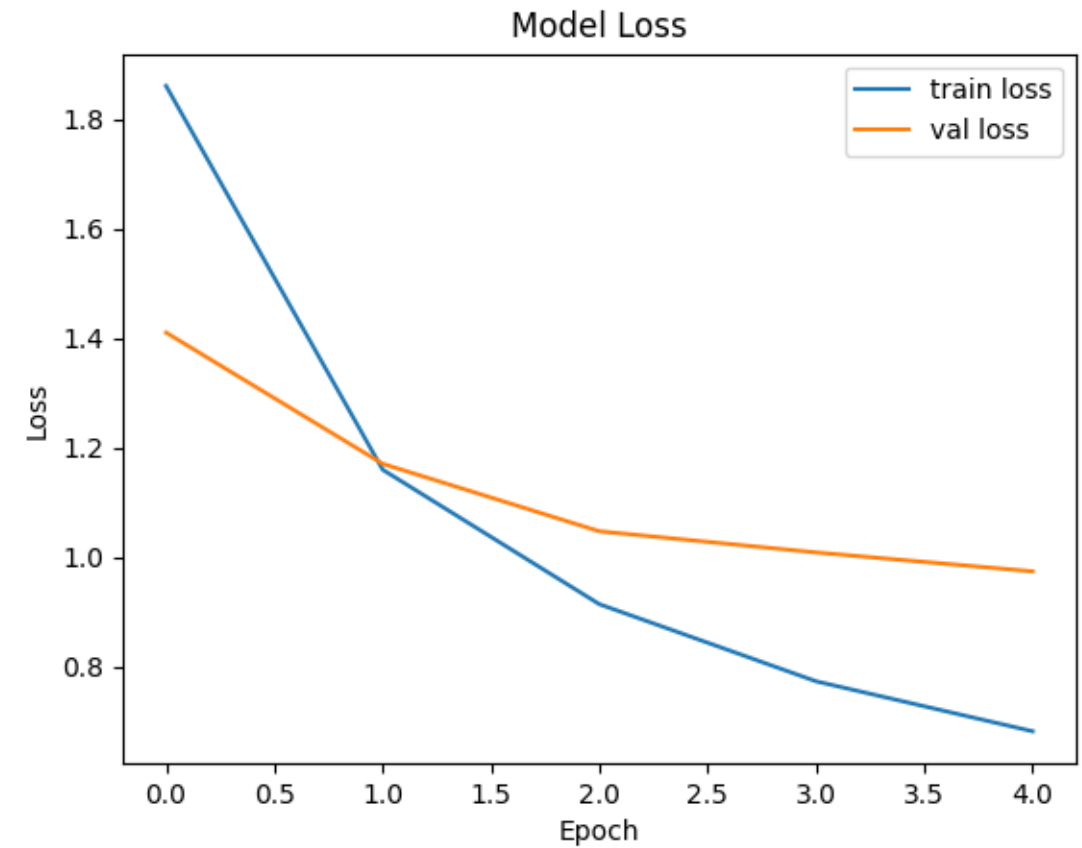
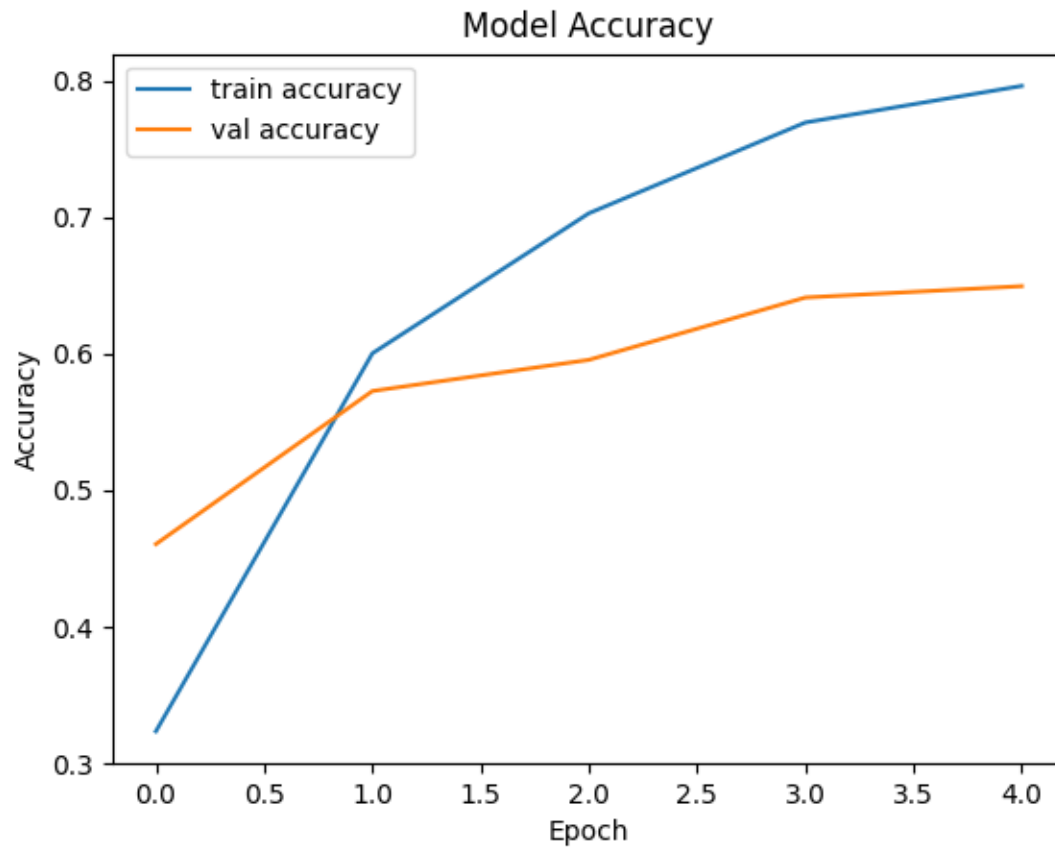
- Building the convolutional neural network (CNN) model,
- Training and optimizing the model's weights,
- Predicting classes for images.

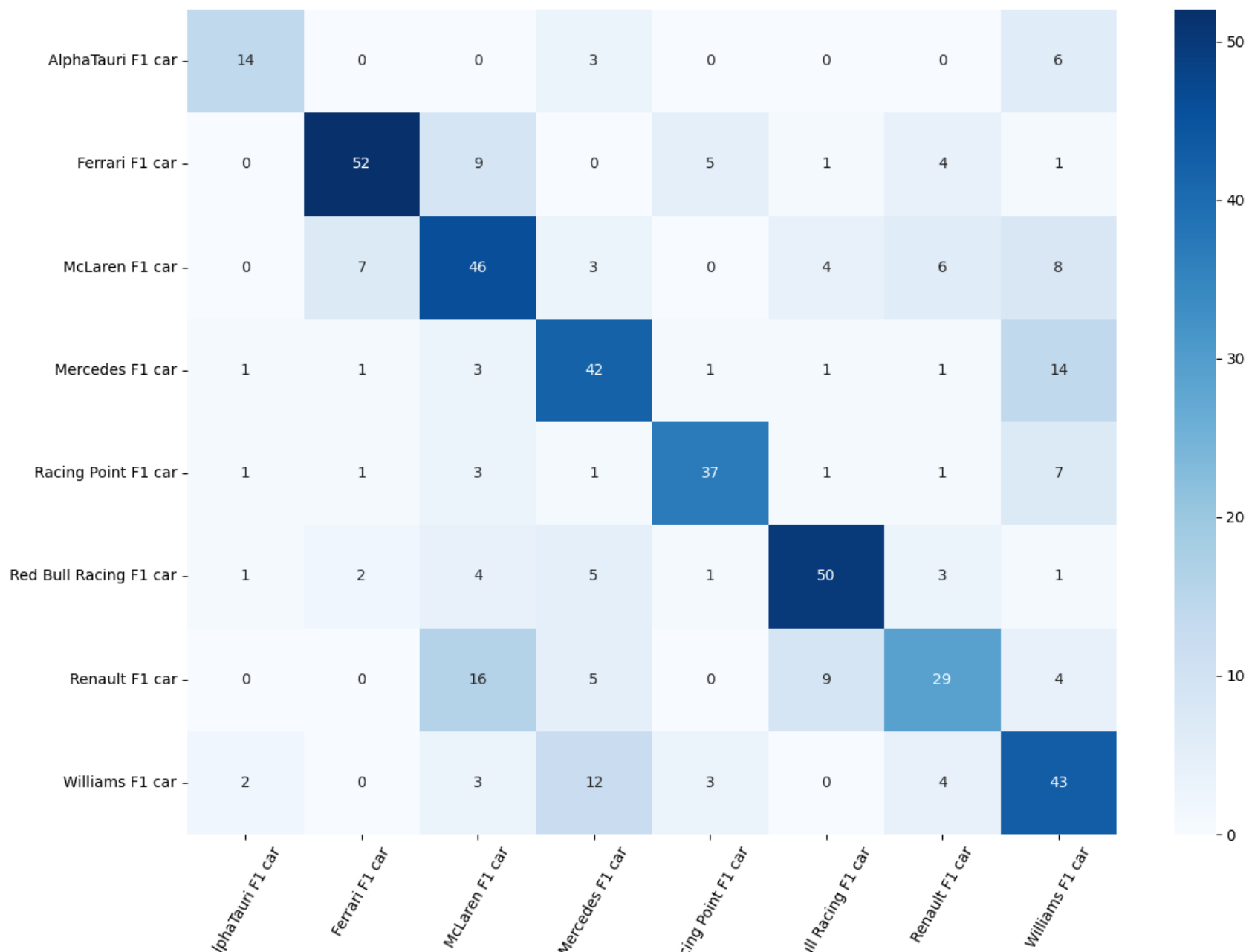
Libraries

- tensorflow.keras.applications

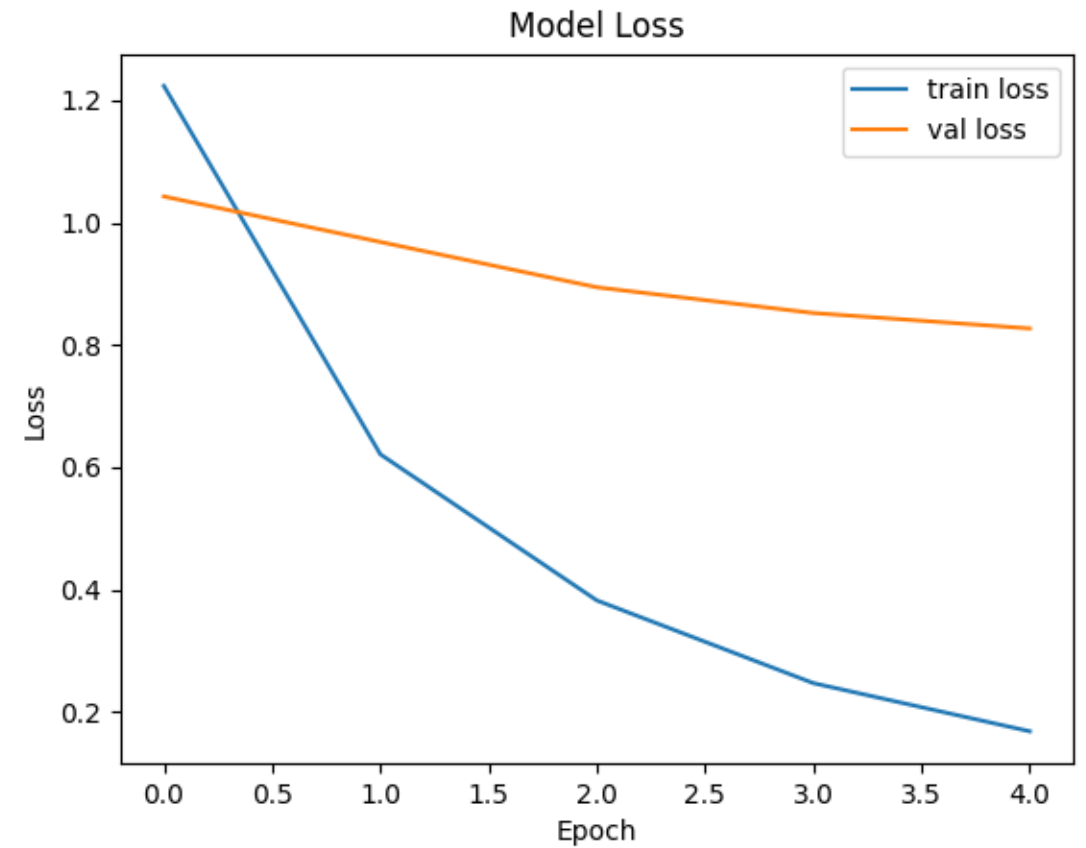
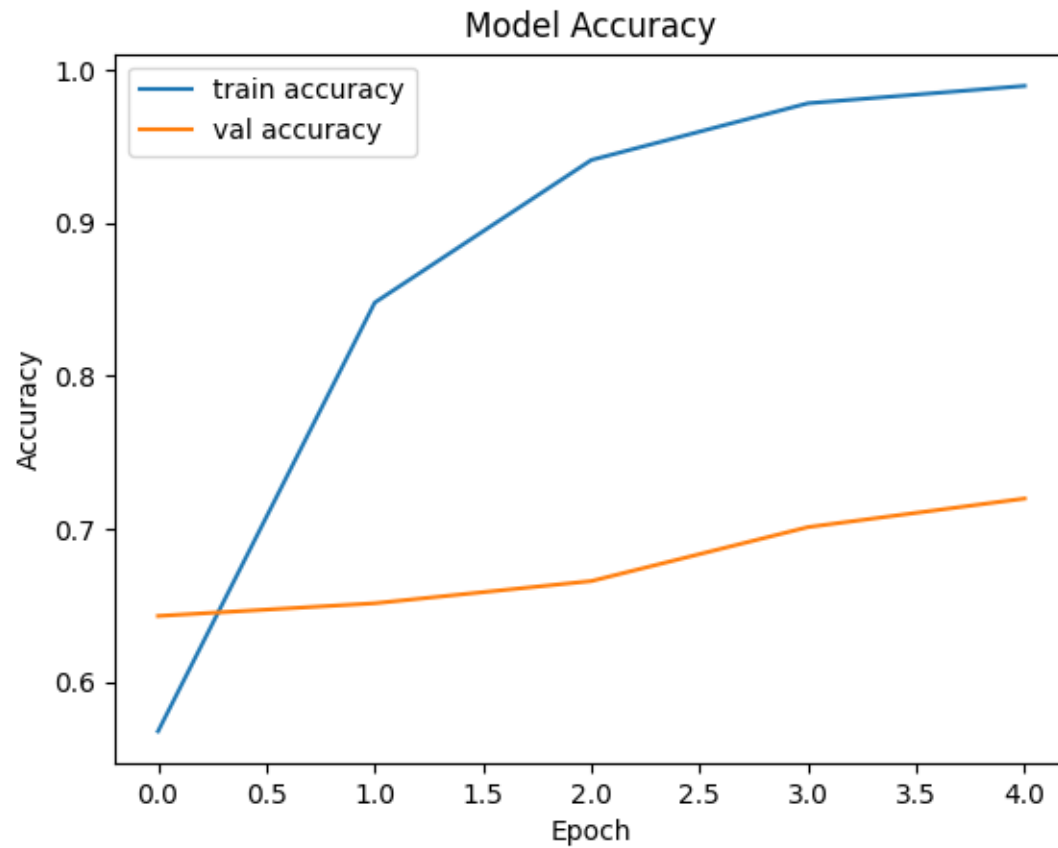
Provides ready-made CNN architectures. These models are often pre-trained on the ImageNet dataset, which allows the use of transfer learning — that is, transferring already learned knowledge to a new task.

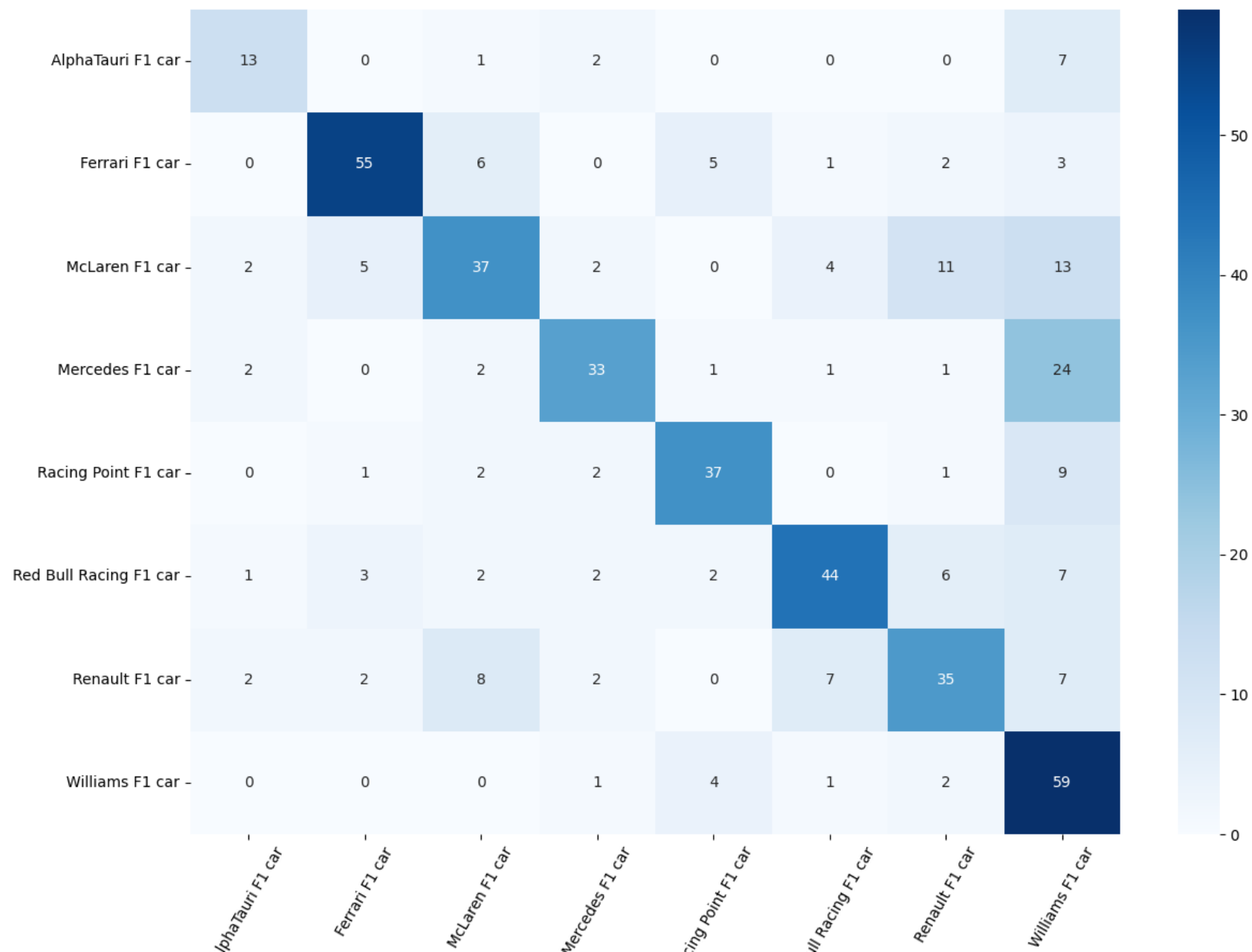
ResNet50V2 before Fine-Tuning



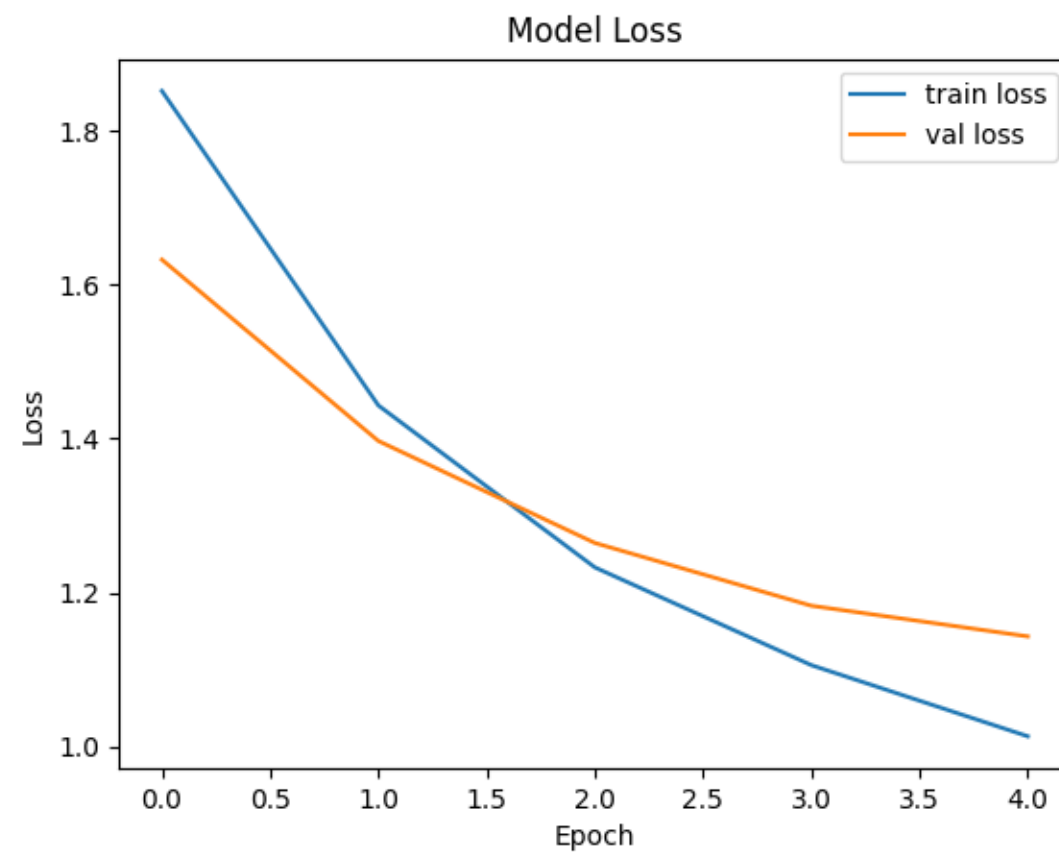
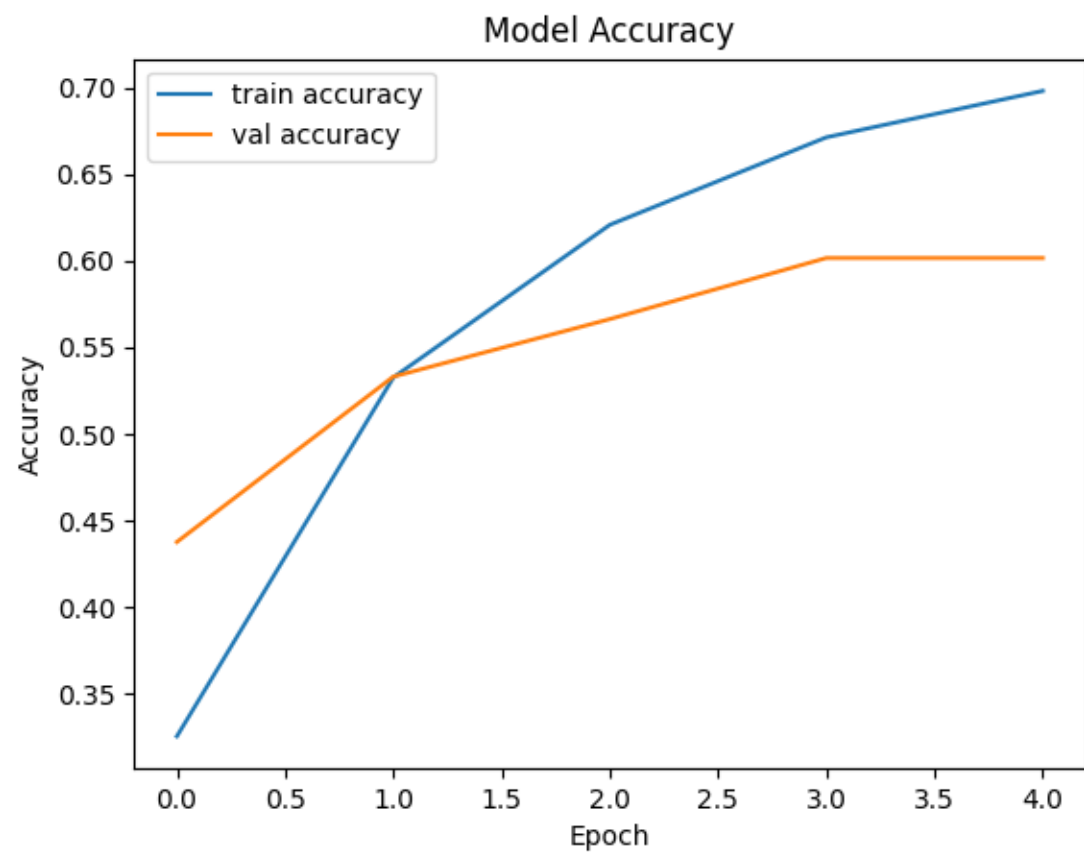


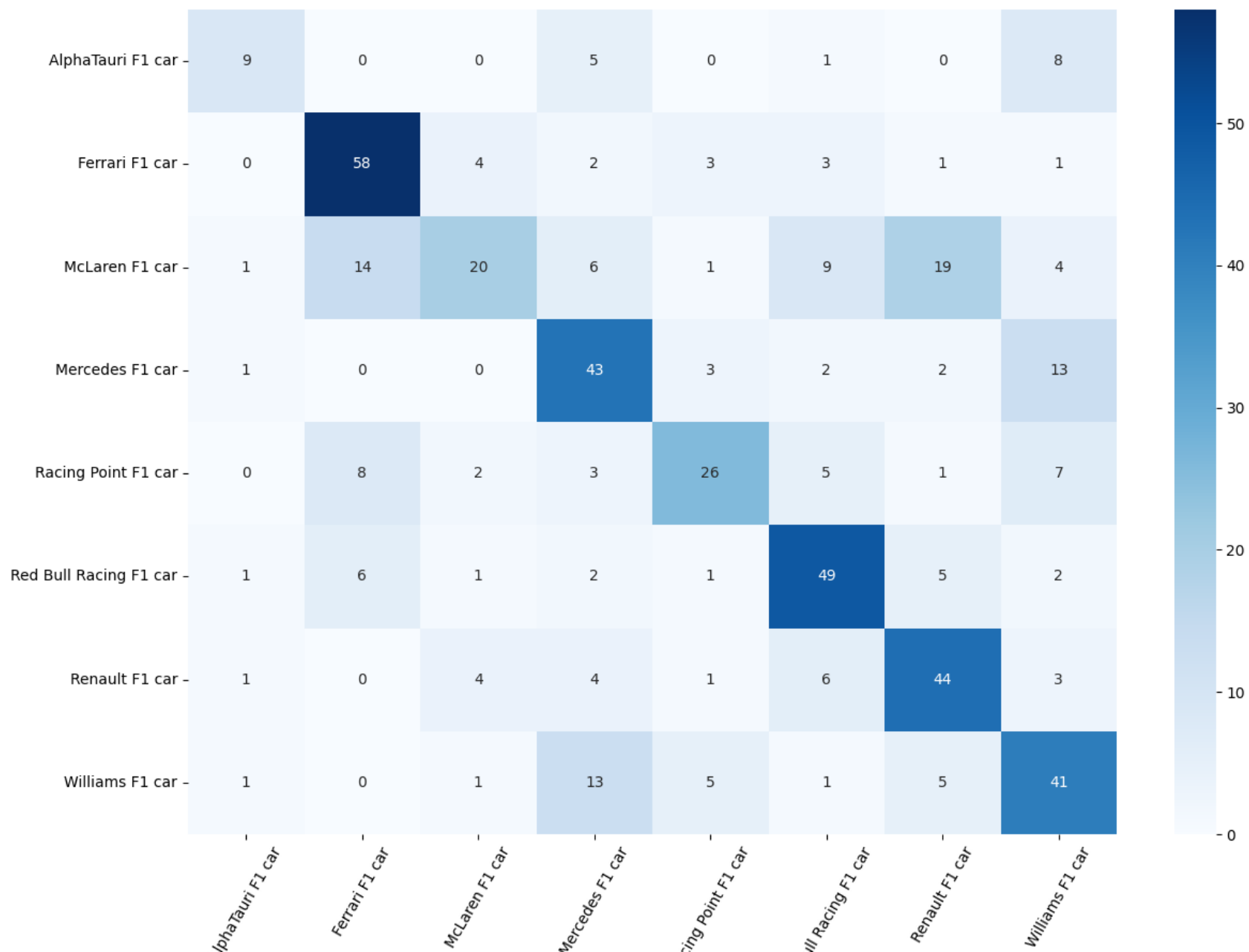
ResNet50V2 after Fine-Tuning



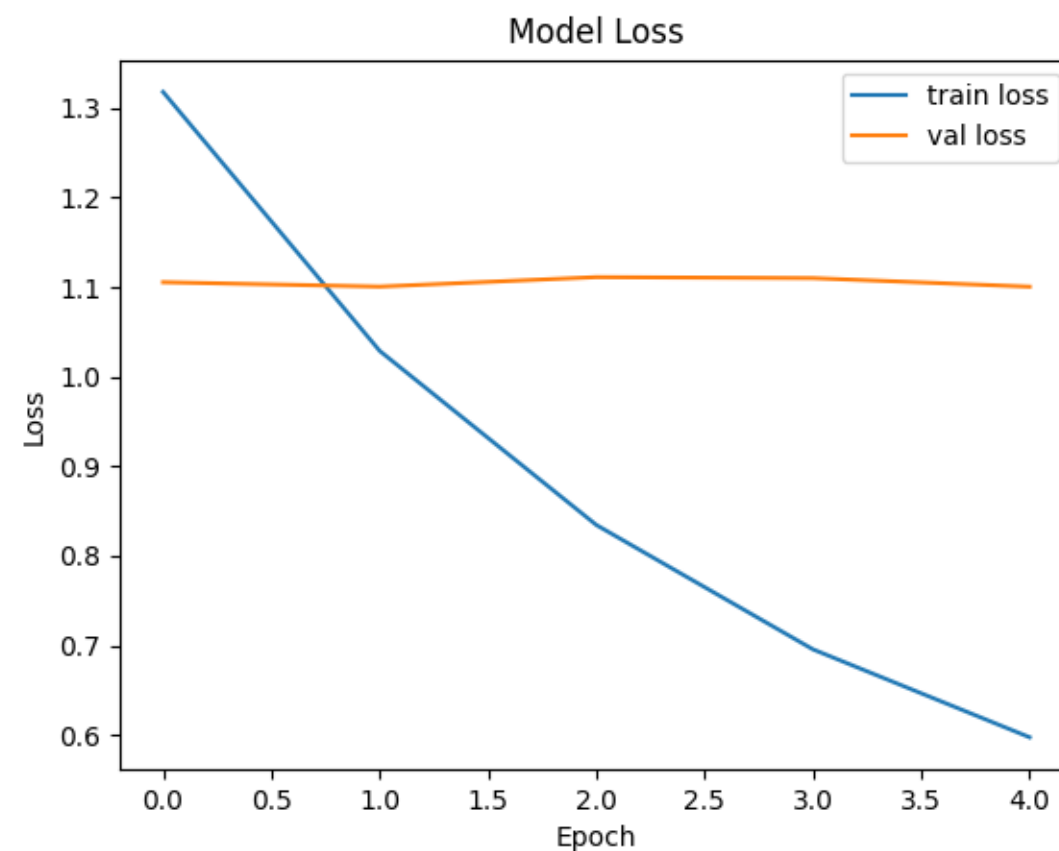
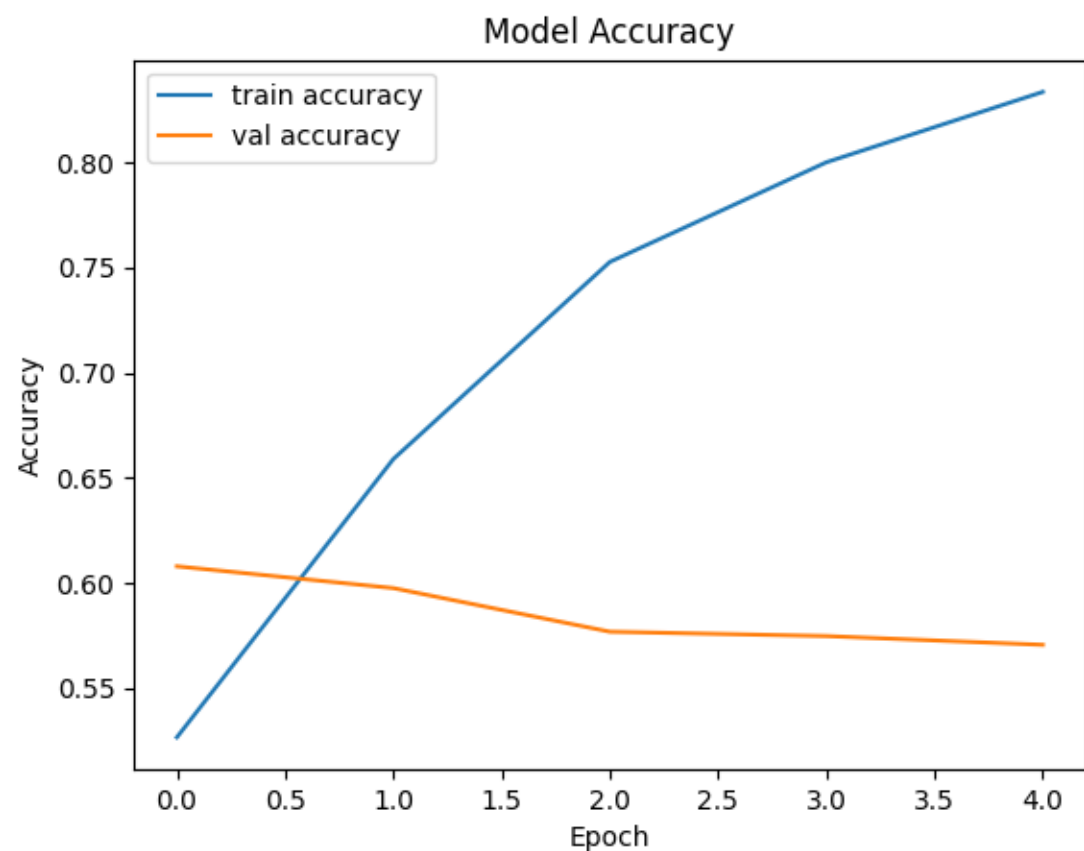


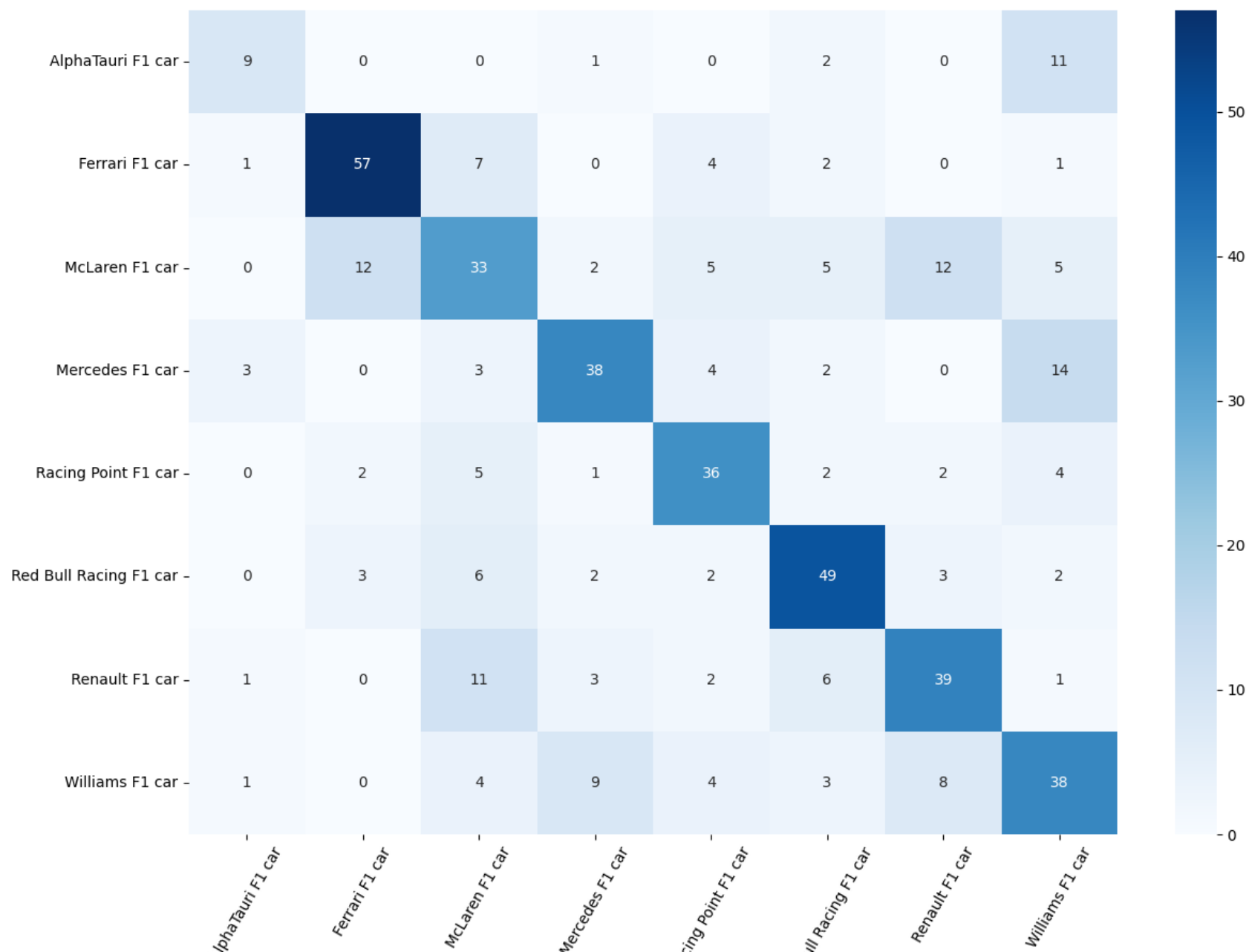
NASNetMobile before Fine-Tuning



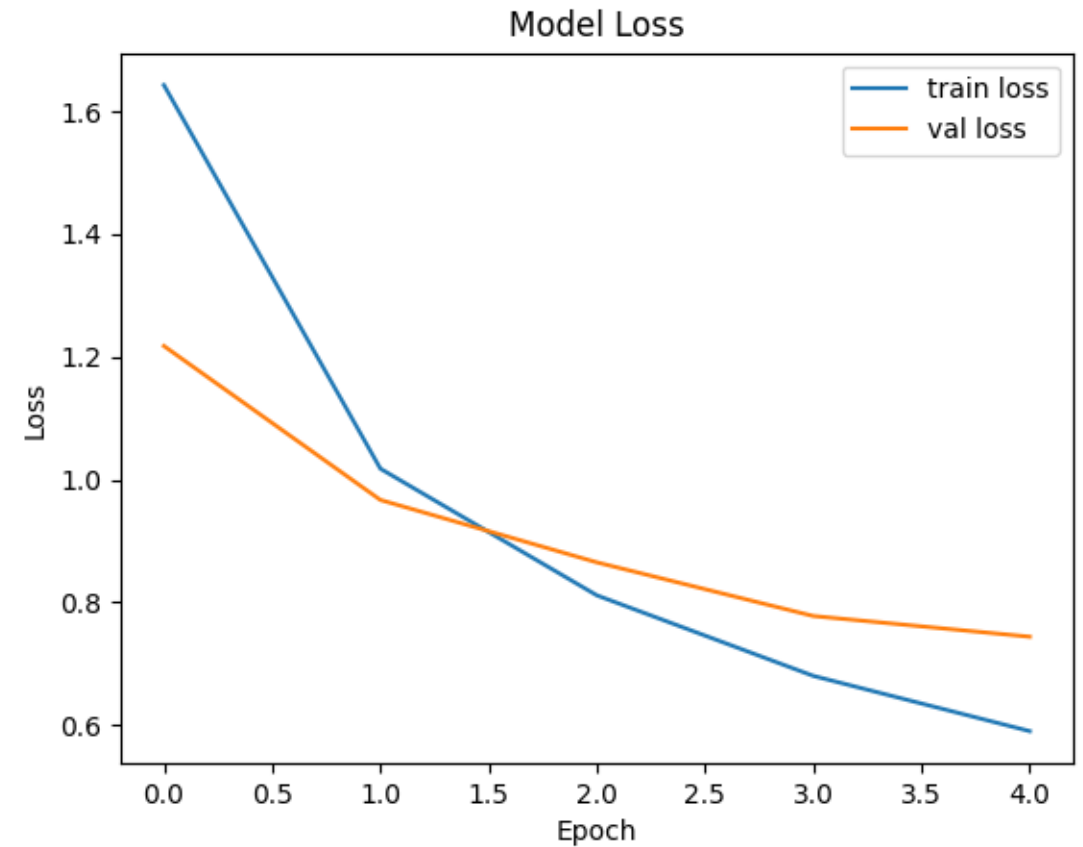
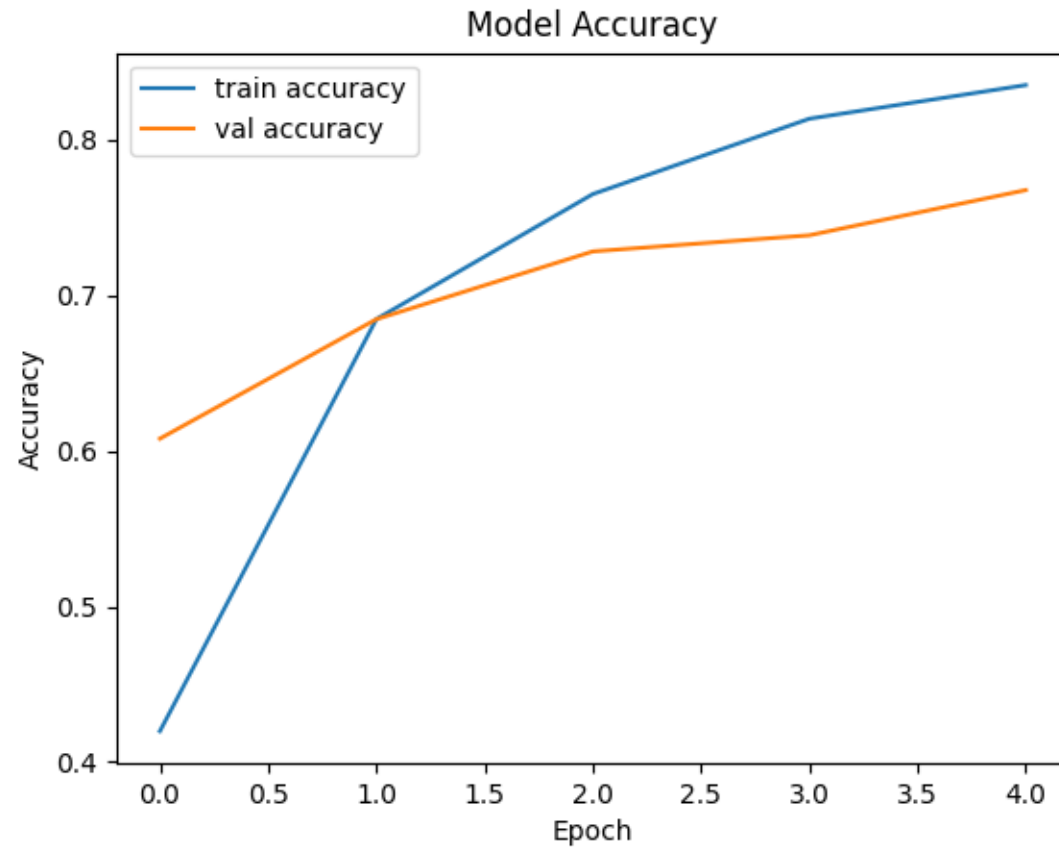


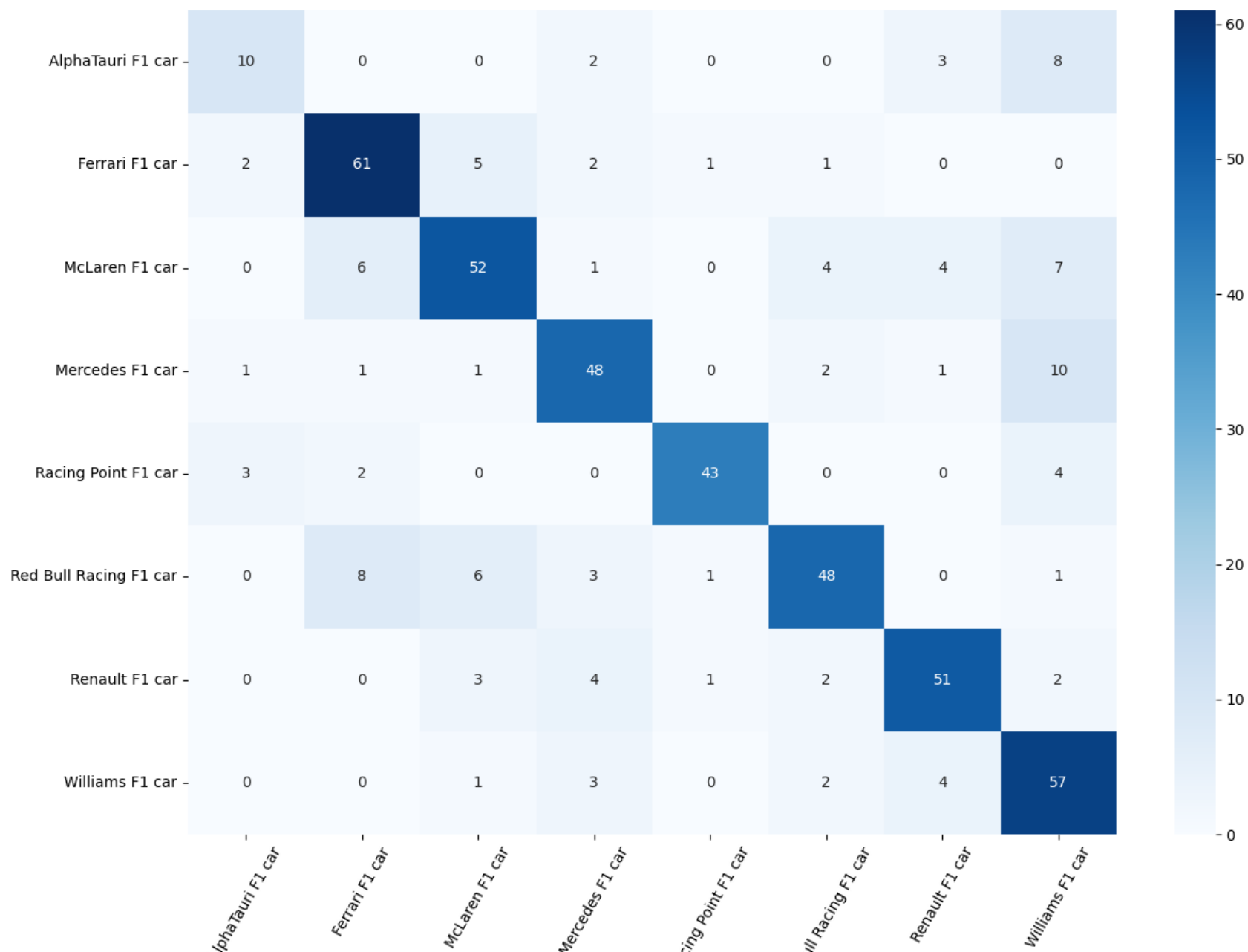
NASNetMobile after Fine-Tuning



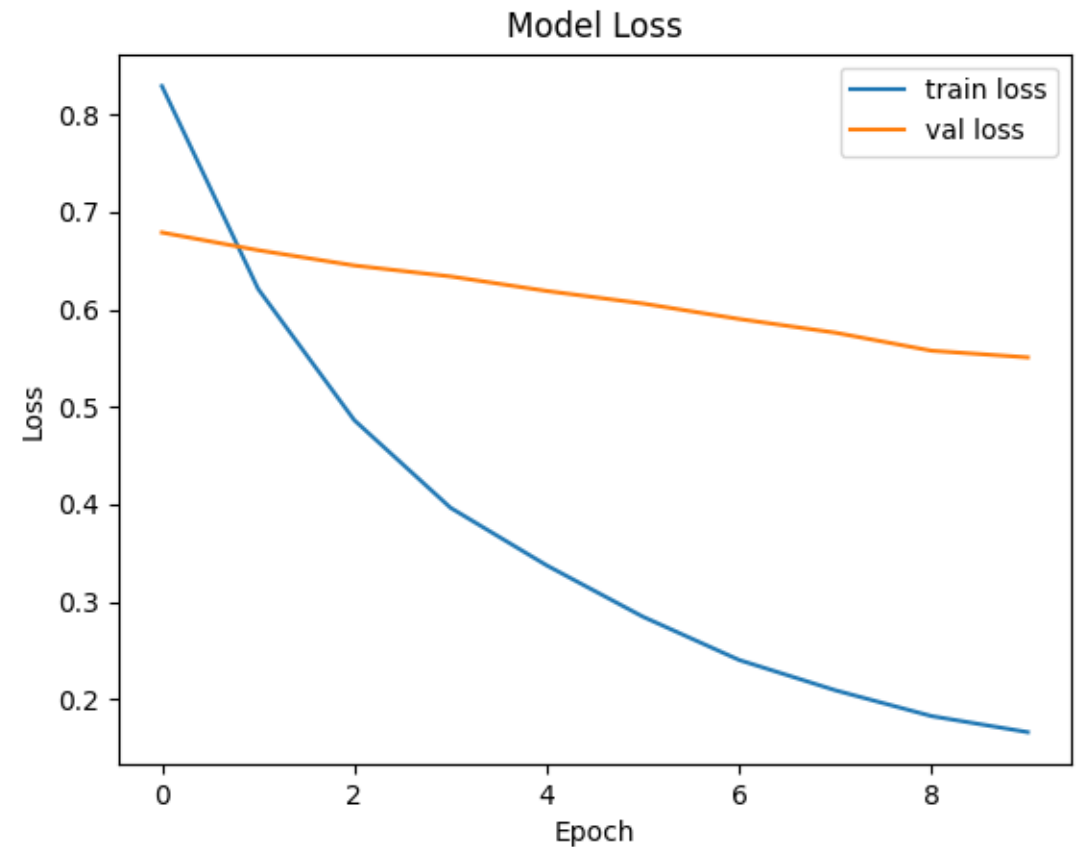
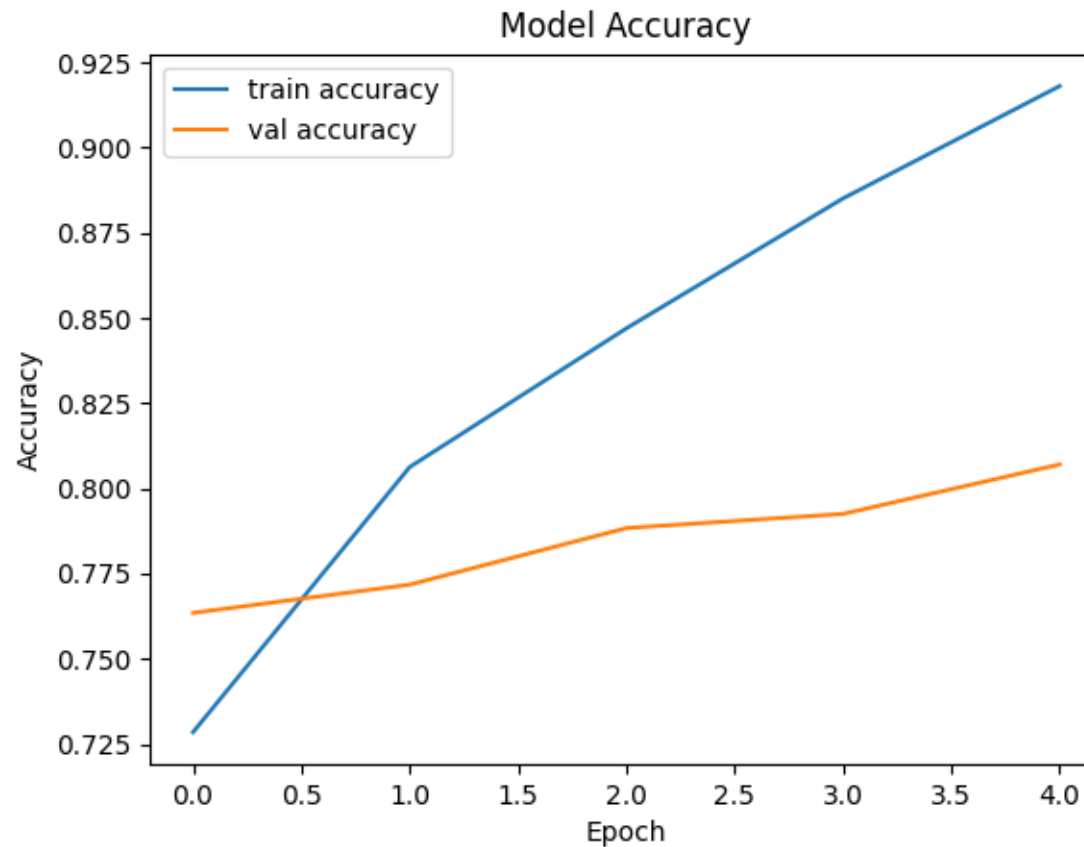


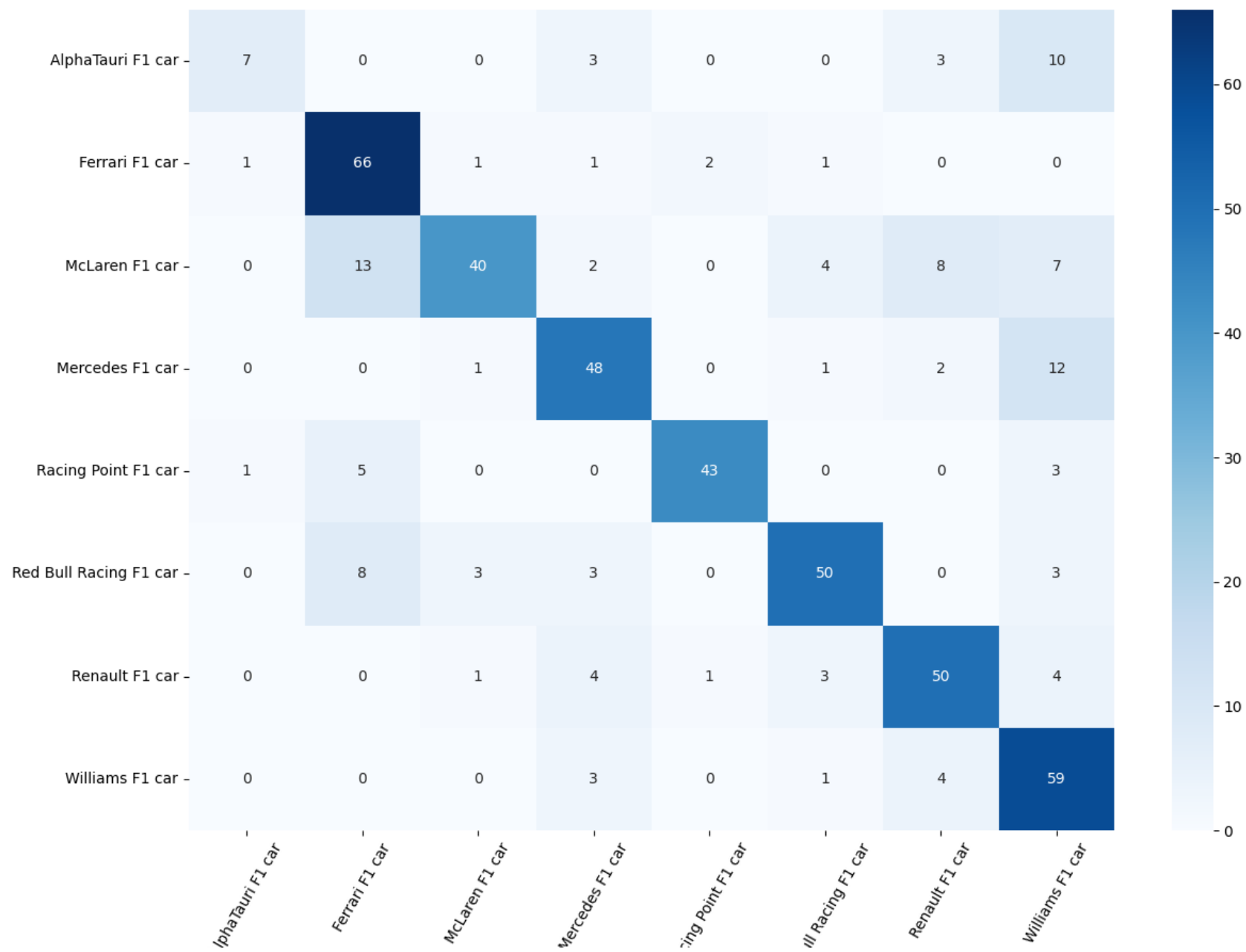
MobileNetV2 before Fine-Tuning





MobileNetV2 after Fine-Tuning





Red Bull Racing F1 car (41.9%)



Red Bull Racing F1 car (96.0%)



models.Sequential

```
model = models.Sequential([
    layers.Conv2D(16, 3, padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(),

    layers.Conv2D(32, 3, padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(),

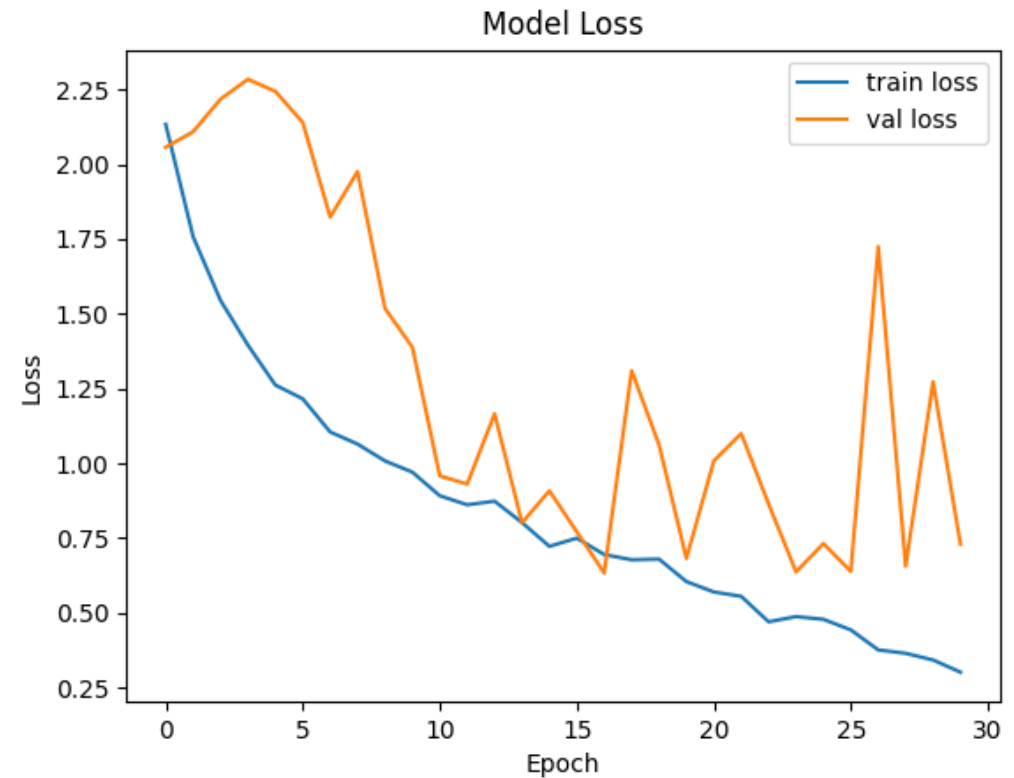
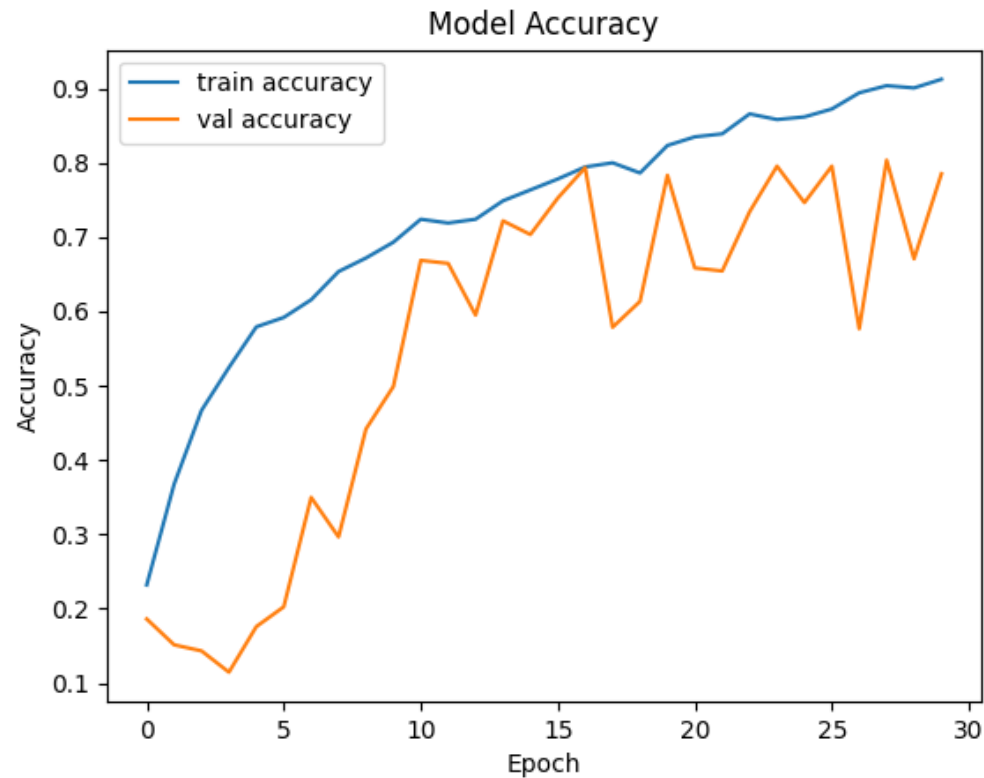
    layers.Conv2D(64, 3, padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(128, 3, padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(),

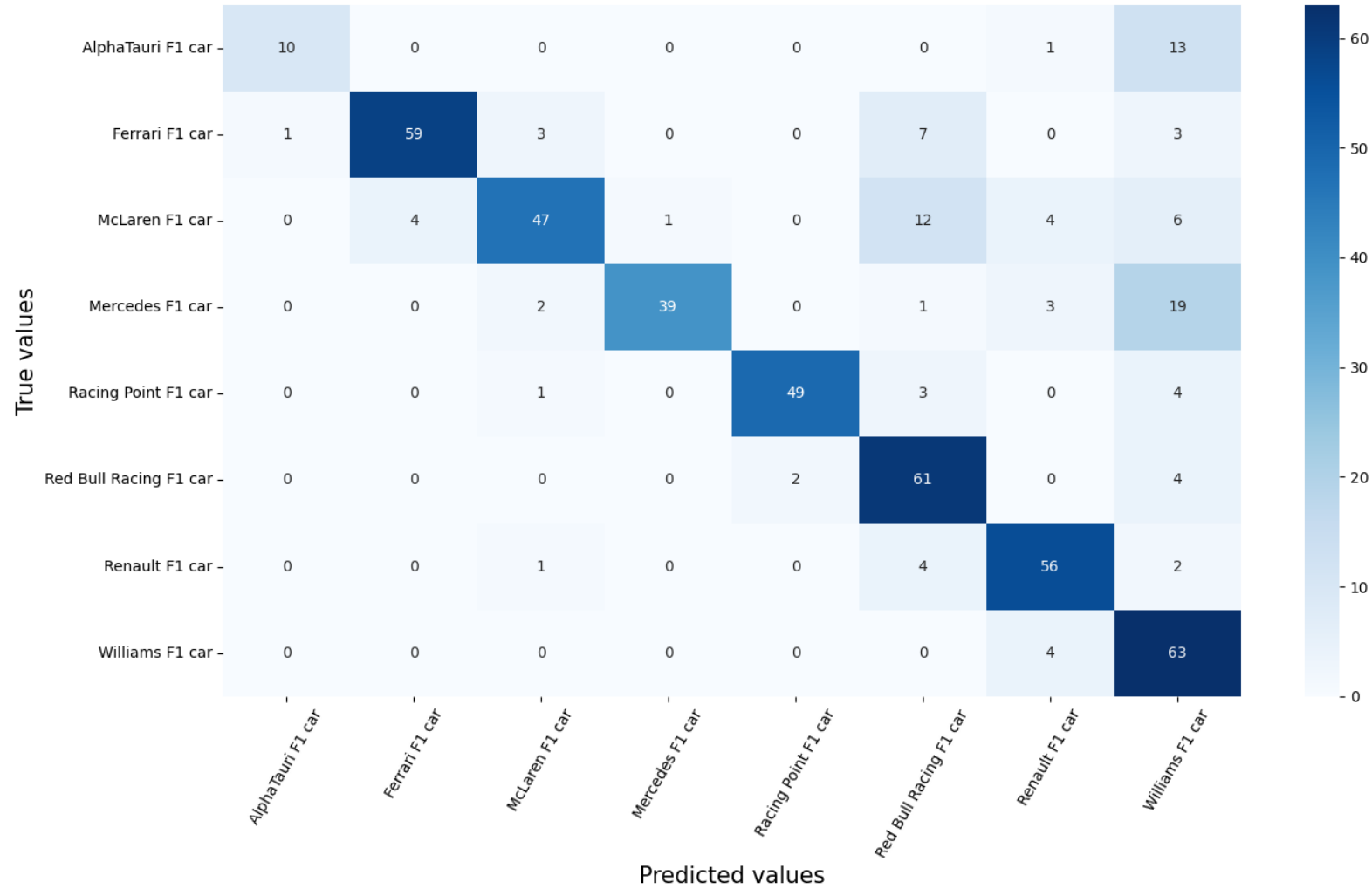
    layers.GlobalAveragePooling2D(),
    layers.Dense(224, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(num_classes, activation='softmax')
])
```

- Input – 224x224x3
- 4 convolutional layers
- BatchNormalization in every layer
- Dropout

Model accuracy and loss over epochs



Confusion matrix

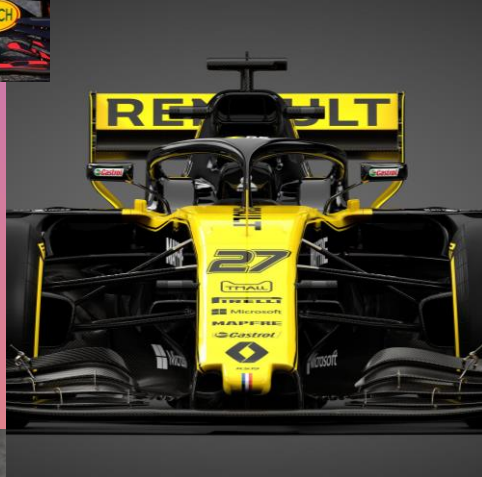


Why didn't some models work?

- Some models are designed for large datasets
- They can be sensitive to the learning rate
- Images were not properly scaled
- They required more advanced preprocessing and fine-tuning

Conclusions

- Fine-tuning significantly improved the model's learning ability (the model learned to recognize patterns more accurately), but it also increased the risk of overfitting.
- Classes with similar appearances or a similar number of examples are harder to distinguish.



THANKS!